London Underground “What-If” Simulation

Full Testing Report

Automated test run: 95 tests passing, 1 skipped / Manual tests: 30 / 30 Pass

1. Key takeaway

The project uses a split testing strategy: automated tests provide fast regression coverage for backend logic, APIs, data processing, train movement calculations and React state hooks, while manual tests verify visual map behaviour, what-if interactions, live train display and mobile responsiveness. All automated tests pass, apart from one intentionally skipped opt-in Neo4j integration test and all 30 manual tests pass.

2. Testing strategy

The test suite is divided into two parts: automated tests for logic-heavy, deterministic code and manual tests for the interactive map-driven UI.

Automated tests cover the backend algorithms, API routes, data-processing utilities, live train pipeline and React state hooks. These tests run quickly and do not rely on external services, so they provide the main regression safety net. Manual tests are used for map rendering, route visualisation, what-if closures, live train behaviour and mobile responsiveness, because these features depend heavily on visual rendering and user interaction.

The split ensures all production logic is regression-tested automatically while all user-facing behaviour is verified manually with photographic evidence.

3. Automated test results

Automated tests cover every pure function and API route in the codebase. This includes the graph construction algorithm, the Dijkstra pathfinding implementation, the TfL data import pipeline, the Neo4j stations API route with its in-memory cache, the full live train pipeline (ID normalisation, stop queue expansion, snapshot building, merging and position rendering), all movement timing and delay utility functions and the core React state hooks. These tests run in milliseconds without any external dependencies and form the primary regression safety net.

All automated tests are run via two commands:

npm test (from the project root), which runs backend suites covering graph, pathfinding, data import and the stations API.

npm test (from frontend/tfl-what-if-simulation), which runs frontend suites covering the train pipeline, utilities, hooks and component tests.

Results: 95 tests pass in total (32 backend + 63 frontend). 1 opt-in Neo4j integration test is skipped by default and requires a live database. One frontend test, useTrainMovements (fallback on unusable arrivals), is timing sensitive and occasionally flaky in constrained CI environments because it relies on asynchronous polling/live-feed state transitions, so this was noted as a residual test stability risk rather than a current failing test; all 5 tests in that suite pass on stable runs.

Test suite	What it covers	Tests	Status	Notes
Backend, Node.js (root npm test) - 5 suites pass, 1 skipped - 32 tests pass, 1 skipped				
graph.test.js	buildGraph - node creation, bidirectional edges, edge normalisation (string coercion), invalid edge skipping with console warning, self-loop handling, input mutation safety	7	Pass	

pathfinding.test.js	dijkstra - shortest path, same-station routing (zero cost), line-change penalty, closed station avoidance, closed line avoidance, blocked edge avoidance,	13	Pass	

Test suite	What it covers	Tests	Status	Notes
	no-route when all paths removed, empty graph and missing destination			
graph-validation.test.js	Network connectivity via BFS - orphan station detection, full-reachability check from hub, isolated subgraph identification, hub-closure reroute, single-corridor no-route	4	Pass	
data-import.test.js	fetchTfl HTTP client - URL construction with API key, non-OK response error and logging; fetchAllLines shape mapping; buildFromRoutes station deduplication, edge creation, haversine distance, multi-sequence handling, API error propagation	5	Pass	
stations-route.test.js	GET /api/stations - Neo4j record mapping to nodes/edges, BigInt/large-integer conversion, in-memory cache (MISS on first request, HIT on second), cache invalidation on error, Neo4j error recovery on subsequent request	3	Pass	
neo4j.graph.test.js	Live Neo4j database integrity - confirms Waterloo station node exists in the populated graph. Opt-in only: requires RUN_NEO4J_TESTS=true and a running local Neo4j instance	1	Skipped	Opt-in integration test. Skipped in default CI run.
Frontend, Next.js (frontend npm test) - 18 suites pass - 63 tests pass				
trainGraphUtils.test.js	buildStationNameIndex - normalised name-to-ID mapping, first-match deduplication; buildEdgeIndexes - directed/reverse travel time map, inbound lookup, per-line edge list, duplicate edge removal, default travel time floor; chooseFromStop - location text parsing, ETA-based inbound selection	3	Pass	
trainRouteExpansion.test.js	findLinePath - line-scoped Dijkstra, bidirectional edges, travel-time cost; expandStopQueue - inserts estimated intermediate stations between TfL predictions with interpolated ETA and arrival timestamps	2	Pass	

trainSnapshotBuilder.test.js	buildLiveSnapshots - vehicle grouping, stop ID normalisation, invalid arrival filtering, stop deduplication; mergeLiveSnapshots - ETA rewind capping (+45s), from-station stability, stale snapshot carryover within window	3	Pass	
trainPositionBuilders.test.js	buildFallbackTemplates - deterministic template creation for known lines only; buildLiveTrainPositions - active stop selection, interpolated map position, location label; buildFallbackTrainPositions – cyclic simulated movement, ETA calculation	3	Pass	
trainsLiveRoute.test.js	GET /api/trains/live - lineIds query param parsing (split, trim, filter), undefined when param absent, 502 on service error	3	Pass	
trainIdUtils.test.js	normaliseTflStopId - 9400ZZ→940G prefix conversion, ZZLU platform-digit suffix trimming, passthrough for non-TfL IDs, empty input; normaliseStationName - suffix removal, punctuation stripping, lowercase; parseTimestampMs - valid ISO parse, null on invalid/missing	4	Pass	
trainMovementTiming.test.js	formatEtaShort - seconds-only, minute-only, mixed; interpolatePosition - progress clamping at 0 and 1, linear interpolation; estimateRemainingSeconds - expectedArrivalMs path vs capturedAt+eta fallback; estimateStopRemainingSeconds - same for stop objects; estimateActiveStopRemainingSeconds - first-future-stop scan; getEdgeTravelTime - map lookup with minimum floor; getPunctuality - late/early/on-time thresholds; hashString - stable non-negative output	8	Pass	
delayUtils.test.js	getDelaySeverityColor - all defined severity levels and grey fallback; getDelaySeverityLabel - all defined levels and Unknown fallback	3	Pass	

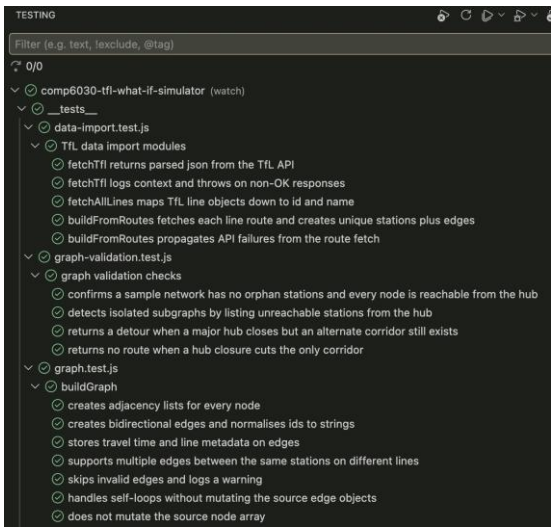
utils.test.js	groupEdges - direction-agnostic pair grouping; buildUndirectedLineEdgeKey - canonical alphabetical key; dedupeEdges - removes reverse duplicates, keeps distinct lines; offsetSegment - perpendicular coordinate offset; buildNodeById / normaliseIdSet - string ID normalisation; splitStateKey - SEP-aware parsing, START→null	6	Pass	
stationMarkerSizing.test.js	Zoom-to-radius mapping consistency, state-based radius boost values, occlusion radius matches visible footprint including stroke width	3	Pass	
useWhatIfClosures.test.js	Ignores station/line toggles when what-if mode is disabled (guard); toggles station and line closure Sets when enabled; handleResetClosures clears both Sets	3	Pass	
useRoutePanel.test.js	Initial state (closed, no route/error); handleRouteChange opens on valid path, closes on no-path; toggleRoutePanel; collapseRoutePanel; clearRoutePanel resets all state	4	Pass	
useLineStatus.test.js	Simulated closures used in what-if mode; live TfL status payload mapping to line/stop closure sets; clean fallback when TfL status API fails	3	Pass	
useTrainMovements.test.js	Disabled in what-if mode with no API polling; graph-not-ready guard; live train positions from usable arrivals; fallback to simulated trains on unusable arrivals; clean fallback on API error	5	Pass	Occasionally has timing flakiness on one test in CI - all 5 tests pass on stable runs.
MapSearchBox.test.js	Enter key selects highlighted station from suggestions; click on suggestion calls onSelectStation	2	Pass	

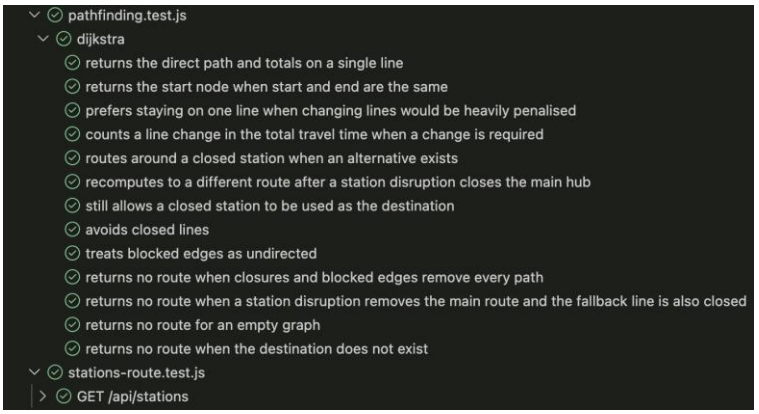
4. Overall summary

Category	Count	Status	Detail
Backend automated tests	32 tests passed, 1 skipped	Pass	5 test suites covering graph building, pathfinding, graph validation, data import, and the stations API route. 1 opt-in Neo4j integration test skipped by default.

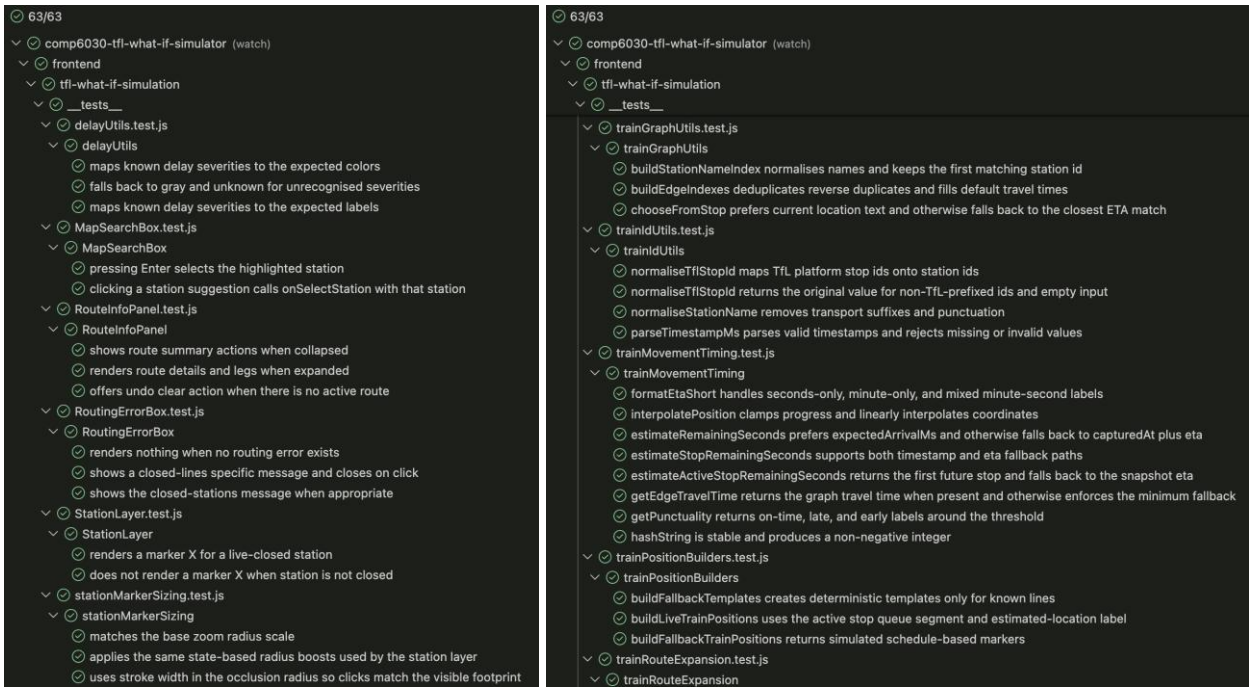
Frontend automated tests	63 tests pass	Pass	18 test suites covering train pipeline logic, ID normalisation, movement timing, map utilities, delay status, all hooks and 4 React UI component tests.
Manual tests	30 of 30 pass	Pass	All 30 manual test cases pass across 7 sections: map load, routing, station closures, line closures, reset, live train feed and mobile / light-theme. Tester: Sattyaj Paul, 28 May 2026, shown below and in a separate manual test plan document which has screenshots
Known issues / limitations	2 observations	Notes	1. useTrainMovements contains one timing-sensitive test that occasionally fails in CI due to async timing, all 5 tests pass on stable runs. 2. MT-21 note: the Reset View button label may be confusing as it also clears the selected route

All automated tests pass. All 30 manual test cases pass. The project meets the testing criteria agreed for the module: logic-heavy backend and utility code is fully covered by automated regression tests, and all user-facing map and UI behaviour has been manually verified with screenshots available in the manual testing document. The one timing-sensitive test in the frontend suite has been noted and does not represent a functional defect.





Above figures show all automated backend test passes.



Above figures show 63/63 of the frontend tests passing.

5. Manual test results

Manual tests cover the Leaflet map rendering, station and route visualisation, the what-if closure flows, live train feed behaviour and mobile responsiveness. These areas are excluded from the automated tests because Leaflet rendering is difficult to test in a headless environment, canvas-based train animation is inherently visual, and the development effort required for a full browser automation suite would be disproportionate to the scale and objectives of this university project.

Tester: Sattyaj Paul - Date: 28 May 2026 - Environment: Chrome, desktop and DevTools mobile viewport - All 30 tests: Pass

The following table records the manual verification evidence. The full document Manual Test Plan with screenshots, is accessible on GitLab

ID	Scenario	Expected result	Actual result	Pass / Fail	Notes / Observations
1. Map load and station render					
MT-01	App loads without errors	Map tiles display. No JS console errors. Station markers visible on the network.	The website loads correctly and connects with the TfL API correctly.	**Pass**	For the website to load correctly with all functionality the TfL API needs to be operational.
MT-02	Station markers render on the correct TfL network	Stations appear at geographically correct positions for the tube network.	The map is accurate. The lines follow the same path as the real map.	**Pass**	The lines could incorporate curves to smooth them to look closer to the real map.
MT-03	Line colours match TfL brand colours	Each line segment is drawn in its official TfL brand colour.	All colours shown on the map and in the sidebar match the official TfL colours.	**Pass**	If a line is closed or partially closed the colour appears as orange to indicate closure.
MT-04	Sidebar / control panel renders and is usable	All UI controls (search, line toggles, what-if panel) are fully rendered with no overflow or cut-off text.	All sidebar controls correctly load, nothing is cut off or overlapping.	**Pass**	A change after a user survey moved the what-if toggle button out of the sidebar, making it easier to spot.
2. Station selection and route display					
MT-05	Select an origin and destination station	A route is drawn on the map between the two stations. Route info panel shows total journey time and any line changes.	The route is correctly drawn with the shortest route. Correct information is shown on the route details.	**Pass**	

MT-06	Route info panel shows correct line-change count	The change count in the panel matches the actual number of interchanges on the visible route.	Correct number of line changes are correctly displayed on the route details.	**Pass**	
MT-07	Route is highlighted / distinct from other lines	The active route is highlighted, and inactive edges are visually de-emphasised.	The rest of the UI dims and makes the highlighted route easier to view.	**Pass**	The highlighted route is clearly shown to the user making it very clear where the journey will go.
MT-07a	Map auto-fits to display the full route on selection	The map animates and zooms to fit the entire route within the viewport, with visible padding. Zoom does not exceed 14.	The viewport is smoothly moved to centre the selected route, and the zoom is correctly adjusted to fit the route.	**Pass**	Some longer routes may be covered by the route details, especially if the route contains multiple line changes.
MT-08	Selecting the same station as origin and destination	App handles gracefully: either shows a zero-time result or prevents selection. No crash or unhandled error.	No crash or error. No route gets created and it returns to the same unselected state.	**Pass**	If the same station is selected, the route details will show the last valid route for the user to go back to.
MT-09	Clearing a selection removes the route	The highlighted route disappears. Route info panel closes or clears.	The interface returns to its usual state with the route details box closing.	**Pass**	The route details box has an undo button which brings back the exact route the user previously selected.
3. What-If Mode - Station closures					
MT-10	Enable what-if mode	What-if mode is active. UI indicates mode change (e.g. toggle highlighted, panel label changes).	Clicking the what-if toggle changes the colour scheme and brings the user to the what-if mode.	**Pass**	Very clear indication of what-if mode with the colour change and the brackets / slow flashing what-if label.
MT-11	Close a non-critical station and verify reroute	Route updates to avoid the closed station. A new route is drawn. Journey time may increase. Route info panel reflects the new path.	Route correctly reroutes and avoids the closed station. Route details are correctly updated to show the new journey.	**Pass**	Like in the live data mode the rest of the UI dims making the selected route clear. The closed station is also clearly shown.
MT-12	Close a hub station and verify reroute or no-route	App finds an alternative route or displays a clear no-route message. Does not crash.	No crash. The program correctly redirects and avoids closed multi-connected stations.	**Pass**	

MT-13	Close origin or destination - no route expected	A no-route or routing error message is shown. No crash. RoutingErrorBox appears with readable text.	Error prompt in red appears clearly stating which stations were selected and that there are no possible reroutes.	**Pass**	The prompt is very clear and obvious to the user that a reroute is not possible.
MT-14	Closed station is visually marked on the map	The closed station is visually distinct from open stations (different icon, colour, or overlay).	Closed stations appear with a red cross and red colour for the station node.	**Pass**	Closed stations may be covered by open stations in busy areas of the map, making them harder to see.
MT-15	Station cannot be closed when what-if mode is off	No closure action occurs. Station is selected/inspected as normal, but closure is not applied.	Closing stations is not possible in the live data mode. The station is just selected and deselected	**Pass**	
4. What-If Mode - Line closures					
MT-16	Toggle a line closure and verify reroute	Route updates to avoid the closed line entirely. The new path uses alternative lines. Route info panel updates.	The new route avoids the closed line and uses other lines.	**Pass**	The newly created route finds the fastest route that skips the closed lines.
MT-17	Close the only viable line - no-route displayed	No-route message displayed. Route info panel or error box shows a readable message. No crash.	No-route message pops up correctly explaining that a line closure is blocking all possible routes.	**Pass**	
MT-18	Closed line is visually de-emphasised on the map	Edges belonging to the closed line are visually distinct (e.g. greyed out, dashed, or hidden).	Closed lines are not de-emphasised, but it is clear which lines are closed as they are dashed.	**Pass**	The lines do lose their distinct colours which helps identify the line when it is partially closed.
MT-19	Close multiple lines simultaneously	Route avoids all closed lines. If no path exists, no-route message is shown. No crash or UI freeze.	Route uses all available lines to reach the destination. If path is not possible the error message appears.	**Pass**	Some lines near the city centre share the exact same path, so there is always a viable route to pick from.
5. Reset Closures					
MT-20	Reset all closures restores original route	All closures are cleared. The original optimal route is restored and displayed. Closed station/line markers return to normal.	Route selects the fastest route after all closures are restored and opened.	**Pass**	

MT-21	Reset when no closures are active is a no-op	No error or crash. UI remains stable. Route (if any) remains unchanged.	Clicking reset repositions the viewport and clears any marked/closed stations.	**Pass**	The reset button is labelled 'Reset View'. This might be confusing for the user as it also resets any selected stations.
6. Live Train Feed					
MT-22	Live train markers appear on the map	Animated train markers appear on line segments, moving between station positions.	Trains appear on the map correctly and follow their designated path along the line.	**Pass**	
MT-23	Train markers show ETA and route label on click	A panel shows: line name, ETA to next station, origin-to-destination label, vehicle ID and punctuality state.	Clicking a train shows all information about the train.	**Pass**	The panel and the train icon are also colour matched with the line the train is currently on.
MT-24	Fallback trains display when live feed is unavailable	Simulated fallback train markers are displayed (moving back and forth along edges). Markers are labelled as simulated / not live.	Fallback trains appear on the TfL map and move between stations.	**Pass**	
MT-25	Train markers filtered by line toggle	Train markers for the deselected line disappear. Markers for other lines remain visible.	Trains of the selected line appear on the map and all other trains disappear.	**Pass**	
MT-26	Train marker density appropriate at different zoom levels	At both zoom levels, the map remains readable. No unacceptable overlap. Performance acceptable.	Zoom in and out to the maximum is smooth. As the map is zoomed out, the map is abstracted.	**Pass**	At the furthest zoom level trains disappear to make the map less cluttered and improve performance across devices.
7. Responsive Mobile layout and Light Theme					
MT-27	App renders correctly on a mobile viewport	Mobile control sheet / bottom drawer is visible. Map occupies the appropriate area. No controls hidden.	All controls are visible in the mobile version of the UI.	**Pass**	Some functions are in a different part of the screen compared to the desktop version due to the smaller screen size.

MT-28	Routing and what-if closures work on mobile layout	All core flows (routing, closure, reset) are usable on the mobile layout without requiring desktop-only controls.	All functionality with routing and rerouting is working.	**Pass**	Although most buttons and features are relocated, everything works as expected and the same as the desktop.
MT-29	Light / dark mode toggle	UI theme switches correctly. All controls, sidebar, route panel, and overlays update. No contrast issues.	UI theme changes to light mode. All overlays are correctly switched, and the font colour changes to black to better contrast.	**Pass**	Dark mode is the default on load.
MT-30	Guided tour - prompt, walkthrough and skip	Tour prompt appears on first load. Stepping through all stops highlights correct elements with readable descriptions. Skipping closes the overlay cleanly.	Prompt clearly guides through the different sections of the interface. Prompt clears after 30 seconds or by clicking No. The tour waits for the user to interact with each feature before advancing.	**Pass**	The tour waits for the user to click and try the feature before moving onto the next item.